

LAPORAN PRAKTIKUM

STRUKTUR DATA

“IMPLEMENTASI SORTING DALAM BAHASA PEMROGRAMAN JAVA”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Sorting dalam Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 5 Juni 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
1.3 Manfaat.....	3
BAB II PEMBAHASAN.....	4
2.1 Pengurutan menggunakan Merge Sort.....	4
2.2.1 Method merge_1002	4
2.1.2 Method sort_1002	5
2.1.3 Method printArray_1002.....	5
2.1.4 Method main	6
2.2 Pengurutan menggunakan Selection Sort	7
2.2.1 Method shellSort_1002	7
2.2.2 Method main().....	8
2.2.2 Method printArray_1002.....	8
2.3 Pengurutan menggunakan Quick Sort	9
2.3.1 Method swap_1002	9
2.3.2 Method medianOfThree_1002	9
2.3.3 Method partition_1002	10
2.3.4 Method quickSort_1002.....	10
2.3.5 Method printArr_1002	11
2.3.6 Method main	11
2.4 Pengurutan menggunakan Bubble Sort menggunakna GUI.....	11
2.5 Latihan Sorting dengan Opsi Memilih Jenis Sorting untuk Mengurutkan Lagu secara Alfabetikal atau Mengurutkan Lagu sesuai Durasi	17
BAB III KESIMPULAN.....	26
3.1 Kesimpulan.....	26
DAFTAR PUSTAKA	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data merupakan salah satu konsep penting dalam struktur data yang digunakan dalam pengolahan informasi. Data yang tidak terurut dapat menyulitkan proses pencarian, analisis, maupun penyajian informasi secara sistematis, sehingga diperlukan algoritma sorting untuk mengatur data agar lebih terstruktur dan efisien.

Berbagai algoritma sorting memiliki karakteristik dan cara kerja yang berbeda, seperti Shell Sort, Quick Sort, dan Merge Sort. Masing-masing algoritma memiliki pendekatan yang berbeda dalam proses pengurutan, baik melalui perbandingan bertahap, pembagian data, maupun penggabungan kembali. Pemahaman perbedaan ini penting untuk mengetahui cara kerja serta kelebihan masing-masing algoritma dalam penerapannya.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma sorting dalam struktur data.
2. Mengimplementasikan beberapa algoritma sorting menggunakan bahasa pemrograman Java.
3. Menganalisis cara kerja Shell Sort, Quick Sort, dan Merge Sort dalam bahasa pemrograman Java.
4. Membandingkan hasil pengurutan berdasarkan algoritma yang digunakan.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menambah pemahaman tentang konsep pengurutan data
2. Melatih kemampuan implementasi algoritma dalam pemrograman Java.
3. Meningkatkan pemahaman logika pemrograman berbasis struktur data.
4. Memahami perbedaan karakteristik beberapa algoritma sorting.

BAB II

PEMBAHASAN

2.1 Pengurutan menggunakan Merge Sort

2.2.1 Method merge_1002

```
1 package pekan8_2511531002;
2
3 public class MergeSort_2511531002 {
4     void merge_1002(int arr_1002[], int l_1002, int m_1002, int r_1002) {
5         int n1_1002 = m_1002 - l_1002 + 1;
6         int n2_1002 = r_1002 - m_1002;
7
8         int L_1002[] = new int[n1_1002];
9         int R_1002[] = new int[n2_1002];
10
11         for (int i_1002 = 0; i_1002 < n1_1002; ++i_1002)
12             L_1002[i_1002] = arr_1002[l_1002 + i_1002];
13         for (int j_1002 = 0; j_1002 < n2_1002; ++j_1002)
14             R_1002[j_1002] = arr_1002[m_1002 + 1 + j_1002];
15         int i_1002 = 0, j_1002 = 0;
16
17         int k_1002 = l_1002;
```

Kode Program 2.1 – Blok Program Method merge

Method ini berfungsi untuk menggabungkan dua bagian array yang sebelumnya sudah diurutkan. Pada tahap ini, array utama dibagi menjadi dua sub-array, yaitu bagian kiri dan bagian kanan, kedua sub-array tersebut kemudian dibandingkan elemen pertamanya satu per satu. Elemen yang memiliki nilai lebih kecil akan dimasukkan terlebih dahulu ke dalam array utama.

```
18     while (i_1002 < n1_1002 && j_1002 < n2_1002) {
19         if (L_1002[i_1002] <= R_1002[j_1002]) {
20             arr_1002[k_1002] = L_1002[i_1002];
21             i_1002++;
22         } else {
23             arr_1002[k_1002] = R_1002[j_1002];
24             j_1002++;
25         }
26         k_1002++;
27     }
28
29     while (i_1002 < n1_1002) {
30         arr_1002[k_1002] = L_1002[i_1002];
31         i_1002++;
32         k_1002++;
33     }
34
35     while (j_1002 < n2_1002) {
36         arr_1002[k_1002] = R_1002[j_1002];
37         j_1002++;
38         k_1002++;
39     }
40 }
```

Kode Program 2.2 – Lanjutan Blok Program Method merge

Proses ini terus dilakukan hingga salah satu sub-array habis. Setelah itu, sisa elemen dari sub-array yang belum habis akan langsung disalin ke array utama. Dengan cara ini hasil penggabungan akan tetap berada dalam keadaan terurut.

2.1.2 Method sort_1002

```
void sort_1002(int arr_1002[], int l_1002, int r_1002) {  
    if (l_1002 < r_1002) {  
        int m_1002 = (l_1002 + r_1002) / 2;  
        sort_1002(arr_1002, l_1002, m_1002);  
        sort_1002(arr_1002, m_1002 + 1, r_1002);  
        merge_1002(arr_1002, l_1002, m_1002, r_1002);  
    }  
}
```

Kode Program 2.3 – Blok Program Method sort

Method ini digunakan untuk melakukan proses pengurutan array dengan metode Merge Sort secara rekursif. Proses dimulai dengan memeriksa apakah indeks kiri masih lebih kecil dari indeks kanan, yang menandakan bahwa array masih dapat dibagi.

Selanjutnya, array dibagi menjadi dua bagian dengan menentukan nilai tengah menggunakan rumus $(l_1002 + r_1002) / 2$. Method ini kemudian memanggil dirinya sendiri untuk mengurutkan bagian kiri dan bagian kanan secara terpisah. Setelah kedua bagian tersebut selesai diurutkan, method **merge_1002()** dipanggil untuk menggabungkan kembali kedua bagian tersebut menjadi array yang sudah terurut.

2.1.3 Method printArray_1002

```
static void printArray_1002(int arr_1002[]) {  
    int n_1002 = arr_1002.length;  
    for (int i_1002 = 0; i_1002 < n_1002; ++i_1002)  
        System.out.print(arr_1002[i_1002] + " ");  
    System.out.println();  
}
```

Kode Program 2.4 – Blok Program Method printArray

Method ini berfungsi untuk menampilkan isi array ke layar. Pada method ini dilakukan perulangan dari indeks pertama hingga indeks terakhir

array. Setiap elemen array kemudian dicetak ke layar dengan pemisah spasi agar mudah dibaca. Method ini digunakan untuk menampilkan kondisi array sebelum proses pengurutan maupun setelah proses pengurutan selesai.

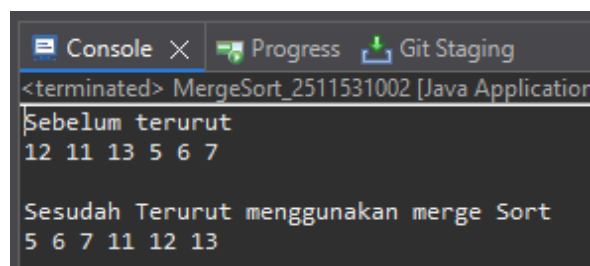
2.1.4 Method main

```
void sort_1002(int arr_1002[], int l_1002, int r_1002) {  
    if (l_1002 < r_1002) {  
        int m_1002 = (l_1002 + r_1002) / 2;  
        sort_1002(arr_1002, l_1002, m_1002);  
        sort_1002(arr_1002, m_1002 + 1, r_1002);  
        merge_1002(arr_1002, l_1002, m_1002, r_1002);  
    }  
}
```

Kode Program 2.5 – Blok Program Method main

Method ini merupakan titik awal eksekusi program. Pada bagian ini, program membuat sebuah array berisi beberapa bilangan acak yang belum terurut. Selanjutnya, array tersebut ditampilkan terlebih dahulu menggunakan method **printArray_1002()** untuk menunjukkan kondisi awal data. Setelah itu, dibuat objek dari kelas **MergeSort_2511531002** untuk memanggil method **sort_1002()** yang berfungsi mengurutkan array menggunakan algoritma Merge Sort.

Setelah proses pengurutan selesai, array kembali ditampilkan untuk menunjukkan hasil akhir yang sudah terurut. Hasil output dari program adalah sebagai berikut:



Gambar 2.1 – Hasil Program Pengurutan menggunakan Merge Sort

2.2 Pengurutan menggunakan Selection Sort

2.2.1 Method shellSort_1002

```
1 package pekan8_2511531002;
2
3 public class ShellSort_2511531002 {
4     public static void shellSort_1002(int[] A_1002) {
5         int n_1002 = A_1002.length;
6         int gap_1002 = n_1002 / 2;
7         while (gap_1002 > 0) {
8             for (int i_1002 = gap_1002; i_1002 < n_1002; i_1002++) {
9                 int temp_1002 = A_1002[i_1002];
10                int j_1002 = i_1002;
11                while (j_1002 >= gap_1002 && A_1002[j_1002 - gap_1002] > temp_1002) {
12                    A_1002[j_1002] = A_1002[j_1002 - gap_1002];
13                    j_1002 = j_1002 - gap_1002;
14                }
15                A_1002[j_1002] = temp_1002;
16            }
17            gap_1002 = gap_1002 / 2;
18        }
19    }
20 }
```

Kode Program 2.6 – Blok Program shellSort

Method ini merupakan implementasi algoritma Shell Sort yang digunakan untuk mengurutkan elemen array secara bertahap berdasarkan jarak tertentu (gap).

Proses dimulai dengan menentukan nilai gap awal yaitu setengah dari panjang array. Selama nilai gap masih lebih besar dari 0, elemen-elemen array akan dibandingkan dan diurutkan menggunakan teknik insertion sort dengan jarak gap tersebut.

Pada setiap iterasi, elemen pada indeks tertentu disimpan sementara ke dalam variabel **temp_1002**, kemudian dibandingkan dengan elemen lain yang berada pada jarak gap di sebelah kiri. Jika elemen sebelumnya lebih besar, maka elemen tersebut akan digeser ke kanan.

Proses ini terus dilakukan hingga posisi yang tepat ditemukan untuk **temp_1002**. Setelah satu putaran selesai, nilai gap akan dibagi dua hingga akhirnya menjadi 0, yang menandakan proses pengurutan selesai.

2.2.2 Method main()

```
21 public static void main(String[] args) {  
22     int[] data_1002 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};  
23  
24     System.out.print("Sebelum: ");  
25     printArray_1002(data_1002);  
26  
27     shellSort_1002(data_1002);  
28  
29     System.out.print("Sesudah: ");  
30     printArray_1002(data_1002);  
31 }
```

Kode Program 2.7 – Blok Program Main

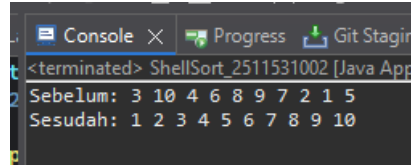
Method ini merupakan titik awal eksekusi program. Pada bagian ini dibuat sebuah array **data_1002** yang berisi beberapa bilangan acak yang belum terurut. Selanjutnya, isi array tersebut ditampilkan ke layar menggunakan method **printArray_1002()** untuk menunjukkan kondisi awal data. Setelah itu, method **shellSort_1002()** dipanggil untuk mengurutkan data menggunakan algoritma Shell Sort. Setelah proses pengurutan selesai, array kembali ditampilkan untuk memperlihatkan hasil akhir yang sudah terurut.

2.2.2 Method printArray_1002

```
21 public static void main(String[] args) {  
22     int[] data_1002 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};  
23  
24     System.out.print("Sebelum: ");  
25     printArray_1002(data_1002);  
26  
27     shellSort_1002(data_1002);  
28  
29     System.out.print("Sesudah: ");  
30     printArray_1002(data_1002);  
31 }
```

Kode Program 2.8 – Blok Program printArray

Method **printArray_1002()** berfungsi untuk menampilkan seluruh elemen array ke layar. Pada method ini digunakan perulangan sederhana untuk mencetak setiap elemen array secara berurutan dengan pemisah spasi, sehingga hasilnya mudah dibaca baik sebelum maupun sesudah proses pengurutan. Hasil output dari program adalah sebagai berikut:



Gambar 2.2 – Hasil Program Pengurutan menggunakan Shell Sort

2.3 Pengurutan menggunakan Quick Sort

2.3.1 Method swap_1002

```

1 package pekan8_2511531002;
2
3 public class QuickSort_2511531002 {
4     static void swap_1002(int[] arr_1002, int i_1002, int j_1002) {
5         int temp_1002 = arr_1002[i_1002];
6         arr_1002[i_1002] = arr_1002[j_1002];
7         arr_1002[j_1002] = temp_1002;
8     }
9

```

Kode Program 2.9– Blok Program Swap

Method ini berfungsi untuk menukar dua elemen pada array. Proses penukaran dilakukan dengan menyimpan salah satu nilai ke dalam variabel sementara (**temp_1002**), kemudian menukar posisi kedua elemen tersebut. Method ini digunakan dalam proses pengurutan untuk memindahkan elemen sesuai posisi yang benar.

2.3.2 Method medianOfThree_1002

```

static void medianOfThree_1002(int[] arr_1002, int low_1002, int high_1002) {
    int mid_1002 = low_1002 + (high_1002 - low_1002) / 2;
    if (arr_1002[low_1002] > arr_1002[mid_1002]) {
        swap_1002(arr_1002, low_1002, mid_1002);
    }
    if (arr_1002[low_1002] > arr_1002[high_1002]) {
        swap_1002(arr_1002, low_1002, high_1002);
    }
    if (arr_1002[mid_1002] > arr_1002[high_1002]) {
        swap_1002(arr_1002, mid_1002, high_1002);
    }
    swap_1002(arr_1002, mid_1002, high_1002);
}

```

Kode Program 2.10 – Blok Program Median of Three

Method ini digunakan untuk menentukan pivot dengan teknik *median of three*, yaitu membandingkan tiga elemen: awal, tengah, dan akhir array. Ketiga elemen tersebut dibandingkan dan diurutkan sehingga didapat

nilai tengah yang lebih representatif sebagai pivot. Setelah itu, pivot dipindahkan ke posisi akhir array agar mempermudah proses partition.

2.3.3 Method partition_1002

```
static int partition_1002(int[] arr_1002, int low_1002, int high_1002) {
    medianOfThree_1002(arr_1002, low_1002, high_1002);
    int pivot_1002 = arr_1002[high_1002];
    int i_1002 = (low_1002 - 1);

    for (int j_1002 = low_1002; j_1002 ≤ high_1002 - 1; j_1002++) {
        if (arr_1002[j_1002] < pivot_1002) {
            i_1002++;
            swap_1002(arr_1002, i_1002, j_1002);
        }
    }
    swap_1002(arr_1002, i_1002 + 1, high_1002);
    return (i_1002 + 1);
}
```

Kode Program 2.11 – Blok Program Partition

Method ini berfungsi untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Elemen yang lebih kecil dari pivot akan ditempatkan di sebelah kiri, sedangkan yang lebih besar di sebelah kanan.

Proses ini dilakukan dengan melakukan perulangan dan menukar elemen yang tidak sesuai posisinya menggunakan method swap_1002(). Setelah selesai, posisi pivot dikembalikan ke tempat yang tepat dan indeks pivot dikembalikan sebagai hasil partisi.

2.3.4 Method quickSort_1002

```
static void quickSort_1002(int[] arr_1002, int low_1002, int high_1002) {
    if (low_1002 < high_1002) {
        int pi_1002 = partition_1002(arr_1002, low_1002, high_1002);
        quickSort_1002(arr_1002, low_1002, pi_1002 - 1);
        quickSort_1002(arr_1002, pi_1002 + 1, high_1002);
    }
}
```

Kode Program 2.12 – Blok Program Quick Sort

Method ini merupakan proses utama dari algoritma Quick Sort yang berjalan secara rekursif. Array akan terus dibagi berdasarkan posisi pivot yang dihasilkan dari method **partition_1002()**. Kemudian, bagian kiri dan kanan dari pivot akan diurutkan kembali menggunakan pemanggilan rekursif sampai seluruh elemen berada dalam kondisi terurut.

2.3.5 Method printArr_1002

```
public static void printArr_1002(int[] arr_1002) {  
    for (int i_1002 = 0; i_1002 < arr_1002.length; i_1002++) {  
        System.out.print(arr_1002[i_1002] + " ");  
    }  
    System.out.println();  
}
```

Kode Program 2.13 – Blok Program Print Array

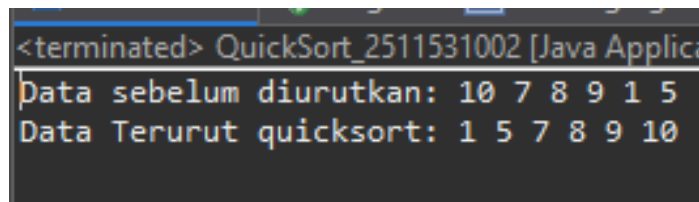
Method ini berfungsi untuk menampilkan isi array ke layar. Method ini melakukan perulangan dari indeks awal hingga akhir array dan mencetak setiap elemen dengan pemisah spasi agar hasil lebih mudah dibaca.

2.3.6 Method main

```
public static void printArr_1002(int[] arr_1002) {  
    for (int i_1002 = 0; i_1002 < arr_1002.length; i_1002++) {  
        System.out.print(arr_1002[i_1002] + " ");  
    }  
    System.out.println();  
}
```

Kode Program 2.14 – Blok Program Print Array

Method ini merupakan titik awal eksekusi program. Pada bagian ini dibuat array berisi data yang belum terurut. Selanjutnya, array ditampilkan terlebih dahulu untuk menunjukkan kondisi awal, kemudian dilakukan pemanggilan method **quickSort_1002()** untuk mengurutkan data menggunakan algoritma Quick Sort. Setelah proses selesai, array kembali ditampilkan untuk memperlihatkan hasil akhir yang sudah terurut. Hasil output dari program adalah sebagai berikut:



```
<terminated> QuickSort_2511531002 [Java Applica  
Data sebelum diurutkan: 10 7 8 9 1 5  
Data Terurut quicksort: 1 5 7 8 9 10
```

Gambar 2. 3 – Hasil Program Pengurutan menggunakan Quick Sort

2.4 Pengurutan menggunakan Bubble Sort menggunakan GUI

```
package pekan8_2511531002;  
  
import java.awt.BorderLayout;  
import java.awt.Color;
```

```

import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511531002 extends JFrame {

    private static final long serialVersionUID = 1L;

    private JPanel contentPane_1002;
    private JTextField inputField_1002;
    private JButton inputButton_1002;
    private JButton stepButton_1002;
    private JButton resetButton_1002;
    private JPanel panelArray_1002;
    private JTextArea stepArea_1002;
    private int[] array_1002;
    private JLabel[] labelArray_1002;
    private int i_1002 = 0;
    private int j_1002 = 0;
    private int stepCount_1002 = 1;
    private boolean sorting_1002 = false;

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> {
            try {
                BubbleSortGUI_2511531002 frame = new BubbleSortGUI_2511531002();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }

    public BubbleSortGUI_2511531002() {
        setTitle("Visualisasi Bubble Sort");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 900, 500);
        contentPane_1002 = new JPanel();
        contentPane_1002.setBorder(new EmptyBorder(10, 10, 10, 10));
        contentPane_1002.setLayout(new BorderLayout(10, 10));
        setContentPane(contentPane_1002);

        JPanel topPanel = new JPanel(new FlowLayout());
        JLabel lblInput = new JLabel("Masukkan angka (pisahkan koma):");
        inputField_1002 = new JTextField(25);
        inputButton_1002 = new JButton("Tampilkan");

        topPanel.add(lblInput);
        topPanel.add(inputField_1002);
        topPanel.add(inputButton_1002);

        contentPane_1002.add(topPanel, BorderLayout.NORTH);
        panelArray_1002 = new JPanel();
        panelArray_1002.setLayout(new FlowLayout());
        contentPane_1002.add(panelArray_1002, BorderLayout.CENTER);
        stepArea_1002 = new JTextArea();
        stepArea_1002.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(stepArea_1002);
        scrollPane.setPreferredSize(new Dimension(300, 0));
    }

```

```

        contentPane_1002.add(scrollPane, BorderLayout.EAST);
        JPanel bottomPanel = new JPanel();

        stepButton_1002 = new JButton("Sorting Selanjutnya");
        resetButton_1002 = new JButton("Reset");
        stepButton_1002.setEnabled(false);
        bottomPanel.add(stepButton_1002);
        bottomPanel.add(resetButton_1002);
        contentPane_1002.add(bottomPanel, BorderLayout.SOUTH);

        inputButton_1002.addActionListener(e -> setArrayFromInput_1002());
        stepButton_1002.addActionListener(e -> nextStep_1002());
        resetButton_1002.addActionListener(e -> reset_1002());
    }

    private void setArrayFromInput_1002() {

        String text = inputField_1002.getText().trim();
        if (text.isEmpty())
            return;
        String[] parts = text.split(",");
        array_1002 = new int[parts.length];
        try {
            for (int k = 0; k < parts.length; k++) {
                array_1002[k] = Integer.parseInt(parts[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(
                this,
                "Masukkan hanya angka yang dipisahkan koma!",
                "Error",
                JOptionPane.ERROR_MESSAGE);
            return;
        }

        i_1002 = 0;
        j_1002 = 0;
        stepCount_1002 = 1;
        sorting_1002 = true;
        stepButton_1002.setEnabled(true);
        stepArea_1002.setText("");
        panelArray_1002.removeAll();
        labelArray_1002 = new JLabel[array_1002.length];

        for (int k = 0; k < array_1002.length; k++) {
            labelArray_1002[k] = new JLabel(String.valueOf(array_1002[k]));
            labelArray_1002[k].setFont(new Font("Arial", Font.BOLD, 24));
            labelArray_1002[k].setOpaque(true);
            labelArray_1002[k].setBackground(Color.WHITE);

            labelArray_1002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_1002[k].setPreferredSize(new Dimension(60, 50));
            labelArray_1002[k].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_1002.add(labelArray_1002[k]);
        }
        panelArray_1002.revalidate();
        panelArray_1002.repaint();
    }

    private void nextStep_1002() {
        if (!sorting_1002)
            return;
        resetColors_1002();
        if (j_1002 < array_1002.length - i_1002 - 1) {
            labelArray_1002[j_1002].setBackground(Color.YELLOW);
            labelArray_1002[j_1002 + 1].setBackground(Color.YELLOW);
            int nilaiKiri = array_1002[j_1002];
            int nilaiKanan = array_1002[j_1002 + 1];
            if (array_1002[j_1002] > array_1002[j_1002 + 1]) {
                int temp = array_1002[j_1002];
                array_1002[j_1002] = array_1002[j_1002 + 1];
                array_1002[j_1002 + 1] = temp;
            }
        }
    }

```

```

        updateLabels_1002();
        labelArray_1002[j_1002].setBackground(Color.RED);
        labelArray_1002[j_1002 + 1].setBackground(Color.RED);
        stepArea_1002.append(
            "Langkah " + stepCount_1002 +
            " : Tukar " +
            nilaiKiri + " dengan " +
            nilaiKanan + "\n");
    } else {
        stepArea_1002.append(
            "Langkah " + stepCount_1002 +
            " : Tidak ada pertukaran (" +
            nilaiKiri + " dan " +
            nilaiKanan + ")\n");
    }
    j_1002++;
} else {
    labelArray_1002[array_1002.length - i_1002 -
1].setBackground(Color.GREEN);
    j_1002 = 0;
    i_1002++;
}
stepCount_1002++;
if (i_1002 >= array_1002.length - 1) {
    sorting_1002 = false;
    for (JLabel lbl : labelArray_1002) {
        lbl.setBackground(Color.GREEN);
    }
    stepButton_1002.setEnabled(false);
    stepArea_1002.append("\n== SORTING SELESAI ==\n");
    JOptionPane.showMessageDialog(this, "Sorting selesai!");
}

private void updateLabels_1002() {
    for (int k = 0; k < array_1002.length; k++) {
        labelArray_1002[k].setText(String.valueOf(array_1002[k]));
    }
}

private void resetColors_1002() {
    if (labelArray_1002 == null)
        return;
    for (JLabel lbl_1002 : labelArray_1002) {
        if (!Color.GREEN.equals(lbl_1002.getBackground())) {
            lbl_1002.setBackground(Color.WHITE);
        }
    }
}

private void reset_1002() {
    inputField_1002.setText("");
    panelArray_1002.removeAll();
    panelArray_1002.revalidate();
    panelArray_1002.repaint();
    stepArea_1002.setText("");
    stepButton_1002.setEnabled(false);
    array_1002 = null;
    labelArray_1002 = null;
    sorting_1002 = false;
    i_1002 = 0;
    j_1002 = 0;
    stepCount_1002 = 1;
}
}

```

Kode Program 2.15 – Kode Program BubbleSortGUI_2511531002

Program ini merupakan implementasi algoritma **Bubble Sort berbasis GUI (Graphical User Interface)** menggunakan bahasa Java. Program ini dikembangkan dengan mengadaptasi **base dari praktikum pekan sebelumnya, yaitu Insertion Sort GUI**, sehingga struktur utama aplikasi seperti input data, tampilan array berbentuk label, serta sistem visualisasi langkah tetap menggunakan pola yang sama, namun dengan perubahan pada logika proses pengurutannya.

Pada awalnya, pengguna memasukkan sekumpulan angka yang dipisahkan dengan tanda koma melalui `inputField`. Data tersebut kemudian diproses dan disimpan ke dalam array, lalu ditampilkan dalam bentuk label pada panel GUI. Setiap elemen array ditampilkan sebagai kotak agar lebih mudah diamati selama proses sorting berlangsung. Setelah data berhasil dimasukkan, tombol untuk melakukan langkah pengurutan diaktifkan.

Proses Bubble Sort dilakukan secara bertahap melalui tombol “Sorting Selanjutnya”. Pada setiap langkah, program akan membandingkan dua elemen yang berdekatan. Jika elemen kiri lebih besar daripada elemen kanan, maka kedua elemen tersebut akan ditukar, dan perubahan langsung diperbarui pada tampilan GUI. Jika tidak terjadi pertukaran, program hanya menampilkan informasi bahwa elemen tersebut sudah berada pada urutan yang benar. Selain itu, warna elemen juga digunakan sebagai penanda visual, yaitu kuning untuk elemen yang sedang dibandingkan, merah untuk elemen yang mengalami pertukaran, dan hijau untuk elemen yang sudah berada pada posisi akhir.

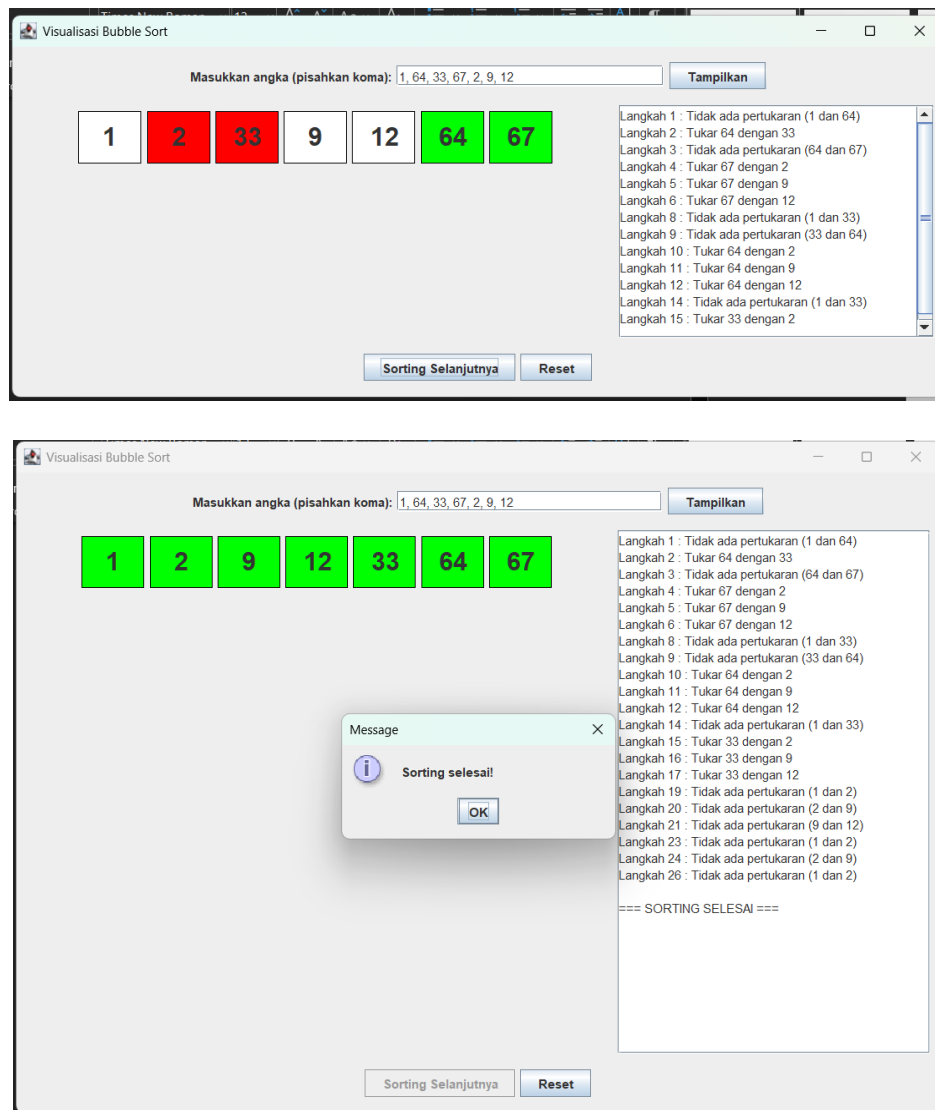
Proses ini berlangsung secara bertahap hingga seluruh elemen berada dalam kondisi terurut. Setiap iterasi memastikan satu elemen terbesar berpindah ke posisi yang benar di bagian akhir array, hingga seluruh data selesai diurutkan. Ketika proses sorting selesai, semua elemen akan ditandai dengan warna hijau dan program akan menampilkan pesan bahwa pengurutan telah selesai.

Selain itu, program juga menyediakan fitur reset yang berfungsi untuk mengembalikan kondisi aplikasi ke keadaan awal. Fitur ini akan

menghapus data yang telah dimasukkan, membersihkan tampilan array, serta mereset seluruh variabel kontrol agar pengguna dapat mengulangi proses dari awal.

Secara keseluruhan, program ini menunjukkan implementasi algoritma Bubble Sort dengan pendekatan visual interaktif, yang dikembangkan berdasarkan struktur dasar dari praktikum sebelumnya, sehingga mempermudah pemahaman alur pengurutan data melalui tampilan grafis.

Hasil Output dari program adalah sebagai berikut:



Gambar 2. 4 – Hasil Program Bubble Sort dengan GUI

2.5 Latihan Sorting dengan Opsi Memilih Jenis Sorting untuk Mengurutkan Lagu secara Alfabetikal atau Mengurutkan Lagu sesuai Durasi

```
package pekan8_2511531002;

public class Lagu_2511531002 {
    String judul_1002;
    String penyanyi_1002;
    int durasi_1002;

    public Lagu_2511531002(String judul_1002, String
    penyanyi_1002, int durasi_1002) {
        this.judul_1002 = judul_1002;
        this.penyanyi_1002 = penyanyi_1002;
        this.durasi_1002 = durasi_1002;
    }
}
```

Kode Program 2.16 – ADT Mahasiswa_2511531002

Program ini merupakan implementasi class **Lagu_2511531002** yang berfungsi sebagai representasi objek lagu dalam sistem sederhana.

Class ini memiliki tiga atribut utama, yaitu **judul_1002** untuk menyimpan judul lagu, **penyanyi_1002** untuk menyimpan nama penyanyi, dan **durasi_1002** untuk menyimpan durasi lagu dalam bentuk bilangan bulat. Ketiga atribut tersebut digunakan untuk mendeskripsikan satu objek lagu secara lengkap.

Untuk menginisialisasi nilai dari atribut-atribut tersebut, digunakan constructor **Lagu_2511531002** yang akan dipanggil ketika objek dibuat. Constructor ini menerima parameter berupa judul, penyanyi, dan durasi, kemudian nilai tersebut langsung disimpan ke dalam atribut class menggunakan keyword **this** untuk membedakan antara variabel parameter dan variabel class.

Secara keseluruhan, class ini berfungsi sebagai blueprint sederhana untuk menyimpan data lagu dalam bentuk objek.

```
package pekan8_2511531002;

import java.util.Scanner;

public class Sorting_2511531002 {
    Lagu_2511531002[] dataLagu_1002 = new Lagu_2511531002[20];
    int jumlahData_1002 = 0;

    // PLAYLIST LAGU
    public void inputData_1002() {
```

```

        dataLagu_1002[jumlahData_1002++] = //Song 1
            new Lagu_2511531002("No One Noticed",
"The Marias", 236);
        dataLagu_1002[jumlahData_1002++] = //Song 2
            new Lagu_2511531002("Best Friend", "Rex
Orange County", 262);
        dataLagu_1002[jumlahData_1002++] = //Song 3
            new Lagu_2511531002("Chicago", "Michael
Jackson", 245);
        dataLagu_1002[jumlahData_1002++] = //Song 4
            new Lagu_2511531002("Made In Japan",
"Buck Owens", 164);
        dataLagu_1002[jumlahData_1002++] = //Song 5
            new Lagu_2511531002("Chest Pain (I
Love)", "Malcolm Todd", 200);
        dataLagu_1002[jumlahData_1002++] = //Song 6
            new Lagu_2511531002("Lovers Rock", "TV
Girl", 213);
        dataLagu_1002[jumlahData_1002++] = //Song 7
            new Lagu_2511531002("NOBLE", "F3miii",
185);
        dataLagu_1002[jumlahData_1002++] = //Song 8
            new Lagu_2511531002("Isn't She Lovely",
"Stevie Wonder", 199);
        dataLagu_1002[jumlahData_1002++] = //Song 9
            new Lagu_2511531002("On the Outside",
"Alicia Goodwyn, Little June", 202);
        dataLagu_1002[jumlahData_1002++] = //Song 10
            new Lagu_2511531002("Akin Ka Na Lang",
"The Itchyworms", 183);
    }

    // MENAMPILKAN DATA
    public void tampilData_1002() {
        for (int i_1002 = 0; i_1002 < jumlahData_1002;
i_1002++) {
            System.out.println(
                (i_1002 + 1) + ". "
                + dataLagu_1002[i_1002].judul_1002
                + " - "
                +
dataLagu_1002[i_1002].penyanyi_1002
                + " - "
                + dataLagu_1002[i_1002].durasi_1002
                + " detik");
        }
    }

    // ALGORITMA SHELL SORT (JUDUL A-Z)
    public void shellSort_1002() {
        for (int gap_1002 = jumlahData_1002 / 2; gap_1002 >
0; gap_1002 /= 2) {
            for (int i_1002 = gap_1002; i_1002 <
jumlahData_1002; i_1002++) {
                Lagu_2511531002 temp_1002 =
dataLagu_1002[i_1002];
                int j_1002;

```

```

        for (j_1002 = i_1002;
              j_1002 >= gap_1002
              &&
dataLagu_1002[j_1002 - gap_1002]

        .judul_1002.compareToIgnoreCase(

        temp_1002.judul_1002) > 0;
                j_1002 -= gap_1002) {
                    dataLagu_1002[j_1002] =
dataLagu_1002[j_1002 - gap_1002];
                }
                dataLagu_1002[j_1002] = temp_1002;
            }
        }
    }

    // ALGORITMA QUICK SORT (DURASI ASC)
    public void quickSort_1002(int low_1002, int high_1002) {
        if (low_1002 < high_1002) {
            int pi_1002 = partition_1002(low_1002,
high_1002);
            quickSort_1002(low_1002, pi_1002 - 1);
            quickSort_1002(pi_1002 + 1, high_1002);
        }
    }

    public int partition_1002(int low_1002, int high_1002) {
        int pivot_1002 =
dataLagu_1002[high_1002].durasi_1002;
        int i_1002 = low_1002 - 1;

        for (int j_1002 = low_1002; j_1002 < high_1002;
j_1002++) {
            if (dataLagu_1002[j_1002].durasi_1002 <=
pivot_1002) {
                i_1002++;

                Lagu_2511531002 temp_1002 =
dataLagu_1002[i_1002];
                dataLagu_1002[i_1002] =
dataLagu_1002[j_1002];
                dataLagu_1002[j_1002] = temp_1002;
            }
        }
        Lagu_2511531002 temp_1002 = dataLagu_1002[i_1002 +
1];
        dataLagu_1002[i_1002 + 1] = dataLagu_1002[high_1002];
        dataLagu_1002[high_1002] = temp_1002;

        return i_1002 + 1;
    }

    // ALGORITMA MERGE SORT (JUDUL A-Z)
    public void mergeSort_1002(int kiri_1002, int kanan_1002) {
        if (kiri_1002 < kanan_1002) {
            int tengah_1002 = (kiri_1002 + kanan_1002) / 2;

```

```

        mergeSort_1002(kiri_1002, tengah_1002);
        mergeSort_1002(tengah_1002 + 1, kanan_1002);

        merge_1002(kiri_1002, tengah_1002, kanan_1002);
    }
}

public void merge_1002(int kiri_1002, int tengah_1002, int
kanan_1002) {
    int n1_1002 = tengah_1002 - kiri_1002 + 1;
    int n2_1002 = kanan_1002 - tengah_1002;

    Lagu_2511531002[] kiriArray_1002 = new
Lagu_2511531002[n1_1002];
    Lagu_2511531002[] kananArray_1002 = new
Lagu_2511531002[n2_1002];

    for (int i_1002 = 0; i_1002 < n1_1002; i_1002++)
        kiriArray_1002[i_1002] =
dataLagu_1002[kiri_1002 + i_1002];
    for (int j_1002 = 0; j_1002 < n2_1002; j_1002++)
        kananArray_1002[j_1002] =
dataLagu_1002[tengah_1002 + 1 + j_1002];

    int i_1002 = 0;
    int j_1002 = 0;
    int k_1002 = kiri_1002;

    while (i_1002 < n1_1002 && j_1002 < n2_1002) {
        if
(kiriArray_1002[i_1002].judul_1002.compareToIgnoreCase(
kananArray_1002[j_1002].judul_1002)
<= 0) {
            dataLagu_1002[k_1002] =
kiriArray_1002[i_1002];
            i_1002++;
        } else {
            dataLagu_1002[k_1002] =
kananArray_1002[j_1002];
            j_1002++;
        }
        k_1002++;
    }

    while (i_1002 < n1_1002) {
        dataLagu_1002[k_1002] = kiriArray_1002[i_1002];
        i_1002++;
        k_1002++;
    }

    while (j_1002 < n2_1002) {
        dataLagu_1002[k_1002] =
kananArray_1002[j_1002];
        j_1002++;
        k_1002++;
    }
}
}

```

```

// MAIN
public static void main(String[] args) {
    Scanner input_1002 = new Scanner(System.in);
    Sorting_2511531002 playlist_1002 = new
Sorting_2511531002();
    playlist_1002.inputData_1002();

    System.out.println("=== Sorting Playlist NIM:
2511531002 ===");
    System.out.print("Pilih Algoritma (1 = Shell, 2 =
Quick, 3 = Merge): ");
    int pilihan_1002 = input_1002.nextInt();

    if (pilihan_1002 < 1 || pilihan_1002 > 3) {
        System.out.println();
        System.out.println("Pilihan tidak valid.");
        input_1002.close();
        return;
    }

    System.out.println("\nData Sebelum Sorting:");
    playlist_1002.tampilData_1002();

    switch (pilihan_1002) {
        case 1:
            playlist_1002.shellSort_1002();
            System.out.println("\nData Setelah Shell
Sort (Judul A-Z):");
            break;
        case 2:
            playlist_1002.quickSort_1002(0,
playlist_1002.jumlahData_1002 - 1);
            System.out.println("\nData Setelah Quick
Sort (Durasi Asc):");
            break;
        case 3:
            playlist_1002.mergeSort_1002(0,
playlist_1002.jumlahData_1002 - 1);
            System.out.println("\nData Setelah Merge
Sort (Judul A-Z):");
            break;
    }
    playlist_1002.tampilData_1002();
    input_1002.close();
}
}

```

Kode Program 2.17 – Class Sorting_2511531002

Class ini merupakan implementasi pengelolaan data playlist lagu yang menggunakan konsep array of object dengan class **Lagu_2511531002** sebagai struktur data utama. Di dalam class ini terdapat array **dataLagu_1002** yang

berfungsi untuk menyimpan objek lagu, serta variabel **jumlahData_1002** yang digunakan untuk menghitung jumlah data yang telah dimasukkan ke dalam array.

Method **inputData_1002()** digunakan untuk mengisi data playlist lagu ke dalam array **dataLagu_1002**. Pada method ini, beberapa objek **Lagu_2511531002** dibuat secara manual dengan berisi informasi judul lagu, nama penyanyi, dan durasi. Setiap objek kemudian dimasukkan ke dalam array secara berurutan, sekaligus meningkatkan nilai **jumlahData_1002** untuk mencatat total data yang telah tersimpan.

Method **tampilData_1002()** digunakan untuk menampilkan seluruh data lagu yang ada di dalam array **dataLagu_1002**. Pada method ini dilakukan perulangan dari indeks awal hingga jumlah data terakhir, kemudian setiap elemen ditampilkan dalam format nomor urut, judul lagu, nama penyanyi, dan durasi dalam detik agar hasilnya mudah dibaca oleh pengguna.

Method **shellSort_1002()** digunakan untuk melakukan pengurutan data lagu berdasarkan judul secara alfabetis (A-Z) menggunakan algoritma Shell Sort. Proses pengurutan dimulai dengan menentukan nilai gap, yaitu setengah dari jumlah data, kemudian secara bertahap diperkecil hingga menjadi 1. Pada setiap tahap, dilakukan perbandingan antar elemen berdasarkan judul menggunakan **compareToIgnoreCase()**, kemudian elemen digeser hingga berada pada posisi yang sesuai.

Method **quickSort_1002()** digunakan sebagai metode utama algoritma Quick Sort yang bekerja secara rekursif untuk mengurutkan data lagu berdasarkan durasi secara ascending. Method ini membagi array menjadi dua bagian berdasarkan hasil dari method **partition_1002()**, kemudian memanggil dirinya sendiri untuk mengurutkan bagian kiri dan kanan hingga seluruh data berada dalam kondisi terurut.

Method **partition_1002()** digunakan untuk menentukan posisi pivot dalam proses Quick Sort. Pivot diambil dari elemen terakhir array berdasarkan durasi lagu. Setelah itu dilakukan perbandingan setiap elemen dengan pivot, di mana elemen yang lebih kecil atau sama akan dipindahkan ke sisi kiri, sedangkan elemen yang

lebih besar berada di sisi kanan. Setelah proses selesai, pivot ditempatkan pada posisi yang tepat dan indeks pivot dikembalikan.

Method **mergeSort_1002()** digunakan untuk melakukan pengurutan data lagu berdasarkan judul secara alfabetis (A-Z) menggunakan algoritma Merge Sort. Prosesnya dilakukan dengan membagi array menjadi bagian kecil secara rekursif hingga hanya tersisa satu elemen, kemudian hasilnya digabungkan kembali dalam keadaan terurut menggunakan method **merge_1002()**.

Method **merge_1002()** digunakan untuk menggabungkan dua sub-array yang sudah terurut menjadi satu array utama. Pada proses ini, elemen dari sub-array kiri dan kanan dibandingkan berdasarkan judul lagu menggunakan **compareToIgnoreCase()**. Elemen yang lebih kecil akan dimasukkan terlebih dahulu ke dalam array utama, hingga semua elemen dari kedua sub-array berhasil digabungkan secara terurut.

Method **main()** digunakan sebagai titik awal eksekusi program. Pada method ini dibuat objek dari class **Sorting_2511531002**, kemudian data lagu diisi menggunakan **inputData_1002()**. Selanjutnya pengguna diberikan pilihan algoritma sorting melalui input, yaitu Shell Sort, Quick Sort, atau Merge Sort. Program kemudian menampilkan data sebelum sorting, menjalankan proses sorting sesuai pilihan, dan menampilkan hasil akhir data yang telah terurut.

Hasil output program akan terlihat seperti berikut:


```
Console X Progress Git Staging
<terminated> Sorting_2511531002 [Java Application] C:\Users\user\.p2\pool\plugins\or
=== Sorting Playlist NIM: 2511531002 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 1
|
Data Sebelum Sorting:
1. No One Noticed - The Marias - 236 detik
2. Best Friend - Rex Orange County - 262 detik
3. Chicago - Michael Jackson - 245 detik
4. Made In Japan - Buck Owens - 164 detik
5. Chest Pain (I Love) - Malcolm Todd - 200 detik
6. Lovers Rock - TV Girl - 213 detik
7. NOBLE - F3miii - 185 detik
8. Isn't She Lovely - Stevie Wonder - 199 detik
9. On the Outside - Alicia Goodwyn, Little June - 202 detik
10. Akin Ka Na Lang - The Itchyworms - 183 detik

Data Setelah Shell Sort (Judul A-Z):
1. Akin Ka Na Lang - The Itchyworms - 183 detik
2. Best Friend - Rex Orange County - 262 detik
3. Chest Pain (I Love) - Malcolm Todd - 200 detik
4. Chicago - Michael Jackson - 245 detik
5. Isn't She Lovely - Stevie Wonder - 199 detik
6. Lovers Rock - TV Girl - 213 detik
7. Made In Japan - Buck Owens - 164 detik
8. No One Noticed - The Marias - 236 detik
9. NOBLE - F3miii - 185 detik
10. On the Outside - Alicia Goodwyn, Little June - 202 detik
```

Gambar 2. 5 – Hasil Program Sorting_2511531002 dengan Shell Sort

```
Console X Progress Git Staging
<terminated> Sorting_2511531002 [Java Application] C:\Users\user\.p2\pool\plugins\or
=== Sorting Playlist NIM: 2511531002 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 2

Data Sebelum Sorting:
1. No One Noticed - The Marias - 236 detik
2. Best Friend - Rex Orange County - 262 detik
3. Chicago - Michael Jackson - 245 detik
4. Made In Japan - Buck Owens - 164 detik
5. Chest Pain (I Love) - Malcolm Todd - 200 detik
6. Lovers Rock - TV Girl - 213 detik
7. NOBLE - F3miii - 185 detik
8. Isn't She Lovely - Stevie Wonder - 199 detik
9. On the Outside - Alicia Goodwyn, Little June - 202 detik
10. Akin Ka Na Lang - The Itchyworms - 183 detik

Data Setelah Quick Sort (Durasi Asc):
1. Made In Japan - Buck Owens - 164 detik
2. Akin Ka Na Lang - The Itchyworms - 183 detik
3. NOBLE - F3miii - 185 detik
4. Isn't She Lovely - Stevie Wonder - 199 detik
5. Chest Pain (I Love) - Malcolm Todd - 200 detik
6. On the Outside - Alicia Goodwyn, Little June - 202 detik
7. Lovers Rock - TV Girl - 213 detik
8. No One Noticed - The Marias - 236 detik
9. Chicago - Michael Jackson - 245 detik
10. Best Friend - Rex Orange County - 262 detik
```

Gambar 2. 6 – Hasil Program Sorting_2511531002 dengan Quick Sort

```
Console X Progress Git Staging
<terminated> Sorting_2511531002 [Java Application] C:\Users\user\.p2\pool\plugins
=== Sorting Playlist NIM: 2511531002 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 3

Data Sebelum Sorting:
1. No One Noticed - The Marias - 236 detik
2. Best Friend - Rex Orange County - 262 detik
3. Chicago - Michael Jackson - 245 detik
4. Made In Japan - Buck Owens - 164 detik
5. Chest Pain (I Love) - Malcolm Todd - 200 detik
6. Lovers Rock - TV Girl - 213 detik
7. NOBLE - F3miii - 185 detik
8. Isn't She Lovely - Stevie Wonder - 199 detik
9. On the Outside - Alicia Goodwyn, Little June - 202 detik
10. Akin Ka Na Lang - The Itchyworms - 183 detik

Data Setelah Merge Sort (Judul A-Z):
1. Akin Ka Na Lang - The Itchyworms - 183 detik
2. Best Friend - Rex Orange County - 262 detik
3. Chest Pain (I Love) - Malcolm Todd - 200 detik
4. Chicago - Michael Jackson - 245 detik
5. Isn't She Lovely - Stevie Wonder - 199 detik
6. Lovers Rock - TV Girl - 213 detik
7. Made In Japan - Buck Owens - 164 detik
8. No One Noticed - The Marias - 236 detik
9. NOBLE - F3miii - 185 detik
10. On the Outside - Alicia Goodwyn, Little June - 202 detik
```

Gambar 2. 7 – Hasil Program Sorting_2511531002 dengan Merge Sort

```
Console X Progress Git Staging
<terminated> Sorting_2511531002 [Java Application] C:\Users\user\.p2\po
=== Sorting Playlist NIM: 2511531002 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 4
|
Pilihan tidak valid.
```

Gambar 2. 7 – Hasil Program Sorting_2511531002 jika salah meng-Input pilihan

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan seluruh praktikum yang telah dilakukan, dapat disimpulkan bahwa sorting merupakan proses penting dalam pengolahan data untuk mengatur data agar lebih terstruktur dan mudah diproses. Pada praktikum ini telah diterapkan beberapa algoritma sorting seperti *Bubble Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort* dengan karakteristik dan cara kerja yang berbeda-beda.

Algoritma *Quick Sort* dan *Merge Sort* bekerja dengan pendekatan rekursif melalui pembagian data menjadi bagian kecil, sedangkan *Shell Sort* melakukan pengurutan bertahap menggunakan gap yang semakin mengecil. Sementara itu, *Bubble Sort* pada implementasi GUI membantu memperjelas proses sorting karena setiap langkah dapat dilihat secara visual.

Selain data sederhana, sorting juga diterapkan pada data berbentuk objek lagu dengan pengurutan berdasarkan judul secara alfabetis dan durasi lagu secara ascending. Hal ini menunjukkan bahwa algoritma sorting dapat digunakan tidak hanya pada data numerik, tetapi juga pada data kompleks dengan atribut tertentu.

Secara keseluruhan, praktikum ini memberikan pemahaman mengenai cara kerja berbagai algoritma sorting serta penerapannya dalam bentuk program Java baik berbasis console maupun GUI.

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 8 – Sorting